



머신러닝 개인 프로젝트

KBO FA 선수의 몸값을 성적 데이터로 예측

시즌 통계 17년치와 직접 만든 FA 계약 210건으로 연평균 계약금을 예측하고, 구단별 적정 제시가까지 산출

데이터 수집

전처리 · 피쳐 설계

모델링

서비스 구현

배포

데모 stovelens-ai.streamlit.app

코드 github.com/jjssspark/SalaryCast_AI

왜 만들었나

PROBLEM

계약의 근거가 어디에도 없음

- 기사에는 **결과 금액만** 나옴. 어떤 성적이 얼마나 반영됐는지 알 수 없음
- 매년 오버페이 논쟁이 붙지만 판단 근거는 결국 **감(感)**
- 정형화된 계약 데이터가 없어 **정답(Y값)부터 직접** 만들어야 했음

SOLUTION

성적 → 금액을 학습하고 근거까지 표시

- FA 직전 **3개년 성적**을 피쳐, **연평균 계약금**을 타겟으로 회귀 학습
- SHAP으로 **몸값을 만든 핵심 요소**를 화면에 그대로 노출
- 구단 조건을 반영한 **구단별 제시가까지** 산출

한눈에 보는 결과

210건

직접 구축한
FA 계약 데이터

17시즌

2010~2026
KBO 선수 통계

136 / 72

학습 표본
타자 / 투수

0.627

타자 모델 R^2
시간 순서 검증

0.477

투수 모델 R^2
시간 순서 검증

배포 완료 Streamlit Community Cloud에 올려 브라우저에서 바로 검색·예측 가능. GitHub Actions로 커밋마다 테스트 자동 실행.

코드 구조 app.py는 라우팅만 담당(102줄). 데이터 로딩 · 피쳐 · 예측 · 카드 조립 · 화면을 `src/` 아래 모듈로 분리해 UI 변경이 예측 로직에 영향을 주지 않게 함.

데이터 파이프라인

공개 데이터셋이 없어 원천 수집부터 직접 구성

1

수집

네이버 스포츠 KBO 시즌 통계.
WAR 정렬만 쓰면 계투·마무리가
누락돼 정렬 기준을 나눠(타자 3개
·투수 4개) 반복 수집 후 중복 제거

requests

2

정답 구축

위키백과 원문(wikitext)에서
계약연도·총액·연수를 뽑아 연평균
계약금 210건으로 정리. 생년이
비거나 틀린 건은 위키데이터
·KBO 기록실로 교정

위키백과 · 위키데이터 · KBO
기록실

3

학습 데이터

계약 직전 3시즌 스탯을 집계하고
타자·투수를 분리. 스타성 지표와
투수 역할 피쳐 추가

타자 136명 · 54피쳐

투수 72명 · 25피쳐

4

학습 · 저장

연봉을 log 변환해 학습하고
모델을 pkl로 저장. 배포 서버는
재학습 없이 로드만

XGBoost

LightGBM

SHAP

모델 성능

표본이 적어 단일 분할은 운에 따라 성능이 요동침 → 과거로만 학습해 다음 해를 맞히는 **시간 순서 검증**을 평가 기준으로 삼음

🔪 타자 · 136명 · 54피처

LightGBM + Ridge 블렌드 · 시장 수준 대비 비율

0.627 **4.45**억

R² (시간 순서)

RMSE



sb_3yr_avg · run_3yr_avg · avg_3yr_avg

⚾ 투수 · 72명 · 25피처

XGBoost + Ridge 블렌드 · 연봉 0.75 + 비율 0.25

0.477 **3.36**억

R² (시간 순서)

RMSE



innings_3yr_avg · war_3yr_avg · star_x_war

베이스라인부터 단계적으로 올림

처음부터 부스팅을 쓰면 왜 이 모델이 더 나은지 수치로 설명할 수 없음

단계	모델	목적	얻은 것
①	선형회귀 / 다중선형회귀	비교 기준선 확보	이후 모든 개선의 기준점
②	RandomForest	비선형 관계 · 피처 중요도	fa_year가 투수 중요도 1위 → 인플레이션을 파고든 계기 (진단은 나중에 뒤집힘)
③	다항 피처 (age_squared)	나이의 비선형 효과	전성기 잔여 구간 피처로 발전
④	블렌드 + 시장 수준 대비 비율 혼합 최종	성능 최대화 + 설명력	타자 R^2 0.627 · 투수 R^2 0.477

데이터 누수를 발견하고 걷어냄

ADR-03 · 성능 수치는 떨어졌지만 그 수치가 비로소 신뢰할 수 있는 값이 됨

문제

초기 데이터셋에 `total_contract_amount` (총액)과 `contract_years` (계약연수)가 그대로 들어 있었음.

총액 = 연평균 계약금 × 계약연수

타겟과 직접 연결된 컬럼이라 모델이 정답을 거의 그대로 베끼고 있었음.

조치

총액 · 계약연수 · 선수명 · 포지션 · 팀 등 타겟을 간접적으로 드러내는 컬럼을 전량 제외하고 데이터셋 버전을 올리며 재구축(현재 계약 v7 · 학습셋 v10).

이후 모든 실험은 누수가 제거된 데이터셋 위에서 진행함.

배운 점 누수가 있으면 검증 점수만 높게 나옴. 실제 예측 시점(계약 전)에는 그 정보가 존재하지 않으므로 모델은 무용지물이 됨.

투수 역할, 모델 3개 대신 피처 1개

ADR-01 · 선발 · 셋업 · 마무리는 중요 스탯이 완전히 다른 (이닝 vs 홀드 vs 세이브)

✕ 역할별 모델 3개로 분리하면

KBO FA 투수는 연간 5~10명 수준.

셋으로 쪼개면 그룹당 표본이 한 자릿수로 줄어 어떤 모델도 패턴을 잡지 못함.

✓ 채택 — `pitcher_role` 단일 피처

모델은 하나만 학습하고 역할을 카테고리 피처로 투입.

트리 계열 모델이 역할별 상호작용을 분기로 알아서 학습함.

분류 규칙 경기당 세이브 0.30 이상 → **CL** · 경기당 홀드 0.25 이상 → **SU** · QS 있거나 경기당 3이닝 이상 → **SP** · 나머지 → **RP**

첫 가설이 틀렸던 문제

TS-005 · 카드가 전부 빈 회색 박스로 깨져 보임

✕ 첫 가설 — 틀림

전역 CSS의 `overflow:hidden` 이 원인이라 추측.

그 규칙은 증상을 가리던 임시방편이었을 뿐 빈 박스를 없애지 못했고, 나중에 호버 팝업이 잘리는 부작용만 냄.

✓ 진짜 원인

Streamlit은 `st.markdown` 호출마다 결과를 독립된 DOM 엘리먼트로 삽입함.

나뉘 쓴 HTML 조각은 남남인 블록이라, 브라우저가 짝 없는 여는 태그를 그 자리에서 바로 닫아버림.

→ 카드 하나를 한 문자열로 조립해 한 번만 호출하도록 수정.

배운 점 파이썬은 오류 없이 돌고 콘솔에도 흔적이 없음. 브라우저에 띄워 눈으로 봐야 발견되는 종류의 버그.

구단별 제시는 모델이 아니라 규칙

선수의 시장 가치와 특정 구단이 부를 금액은 다른 질문이라 분리함

$$\text{offer} = \text{예측연봉} \times (1 + \text{포지션필요도} \times 0.20 + \text{원나우지수} \times 0.15 + \text{샐러리캡여유} \times 0.10)$$

0.20

포지션 필요도

0.15

원나우 지수

0.10

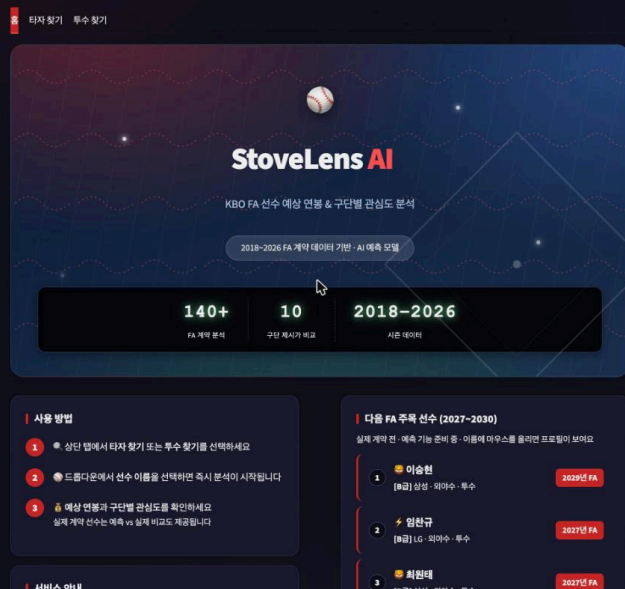
샐러리캡 여유

왜 규칙인가

구단 데이터는 10개 구단 × 포지션별 필요도라 표본이 너무 적어 함께 학습시킬 수 없었음. 규칙으로 빼니 제시가 근거를 사람이 읽을 수 있는 형태로 설명 가능해짐.

서비스 화면 — 검색과 평가

사용자는 야구 팬과 구단 관계자. R^2 · RMSE 같은 용어는 화면에서 전부 건너냄



홈 — 선수 검색

이름 검색과 팀 필터로 선수를 찾고, 다음 FA 주목 선수를 카드로 노출



실제 계약 vs AI 예측

FA가 끝난 선수는 두 금액을 비교해 고평가 · 적정 · 저평가를 문장으로 제시

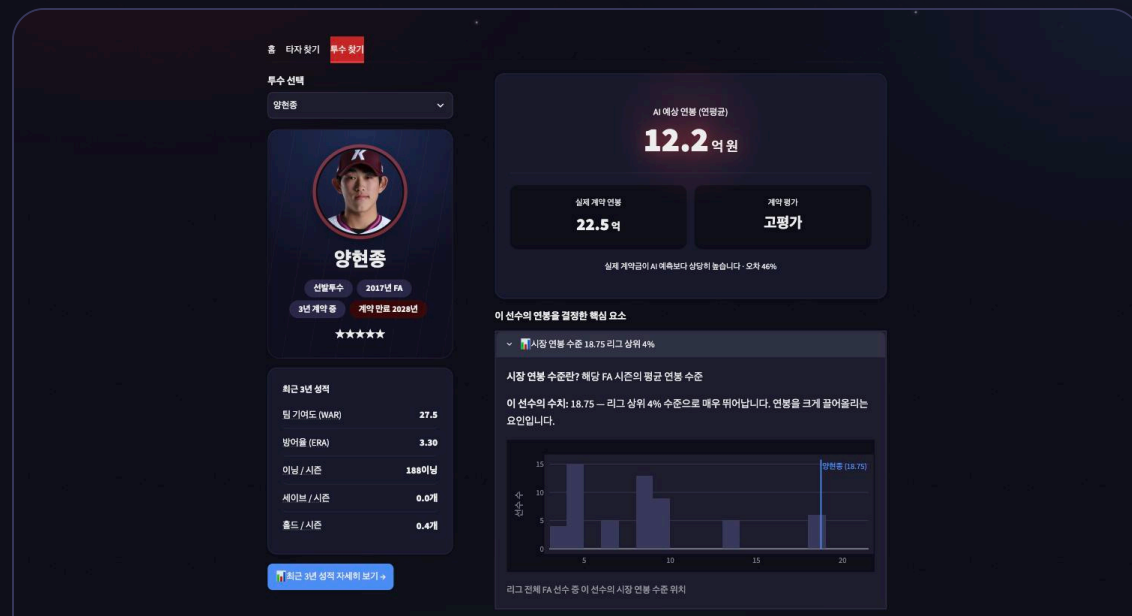
서비스 화면 — 제시가와 근거

기술 지표는 분석 상세 보기 토글 안으로 격리



구단별 예상 제시가

10개 구단의 포지션 필요도 · 재정 · 우승 욕구를 반영한 제시가 범위



예측 근거

몸값에 기여한 핵심 스탯과 그 의미를 함께 표시하고, 용어에는 톨팁 제공

기술 스택

실제로 사용한 것만

언어 · 데이터

Python 3.13

pandas

numpy

수집 · 모델링 · 서비스를 한 언어로 끊김 없이 연결

머신러닝

scikit-learn

XGBoost

LightGBM

SHAP

선형회귀 베이스라인에서 트리 앙상블로 고도화.
SHAP으로 예측 근거를 화면에 설명

웹 서비스

Streamlit

Plotly

별도 API 서버나 프론트엔드 통신 계층 없이 파이썬 코드와 화면을 직결

데이터 수집

requests

네이버 스포츠 API와 위키백과·위키데이터 호출.
위키는 렌더링된 HTML 대신 원문을 정규식으로 읽음

테스트 · CI

pytest

ruff

GitHub Actions

커밋마다 회귀 확인. 별도 인프라 없이 GitHub에서 동작

배포

Streamlit Community Cloud

커밋된 pickle · csv만 읽고 서버에서 재학습하지 않음

한계와 개선 방향

지금 이 모델이 설명하지 못하는 것

표본이 절대적으로 부족

계약을 2014년까지 넓혀 타자 136명 · 투수 72명이 됐지만 여전히 작음. 게다가 네이버가 타자 WAR를 2017년부터, 투수 WAR를 2014년부터만 줘서 그 이전 계약은 최상위 피처 없이 학습에 들어감. 데이터를 늘려 올리는 방향은 거의 소진

성적 외 요인이 없음

구단 마케팅 판단 · 협상력 · 에이전트 · 잔류 프리미엄은 반영되지 않음. 스타성 지표로 일부 보완했으나 완전하지 않음

연봉 인플레이션 — 진단이 반대였음

처음엔 모델이 물가를 학습해 금액을 부풀린다고 봤으나 반대였음. 트리가 학습에 없던 연도를 외삽 못 해 미래를 계속 낮게 불렀음(타자 -1.79억 · 투수 -3.80억). 시장 수준 대비 비율 예측을 섞어 -0.70억 · -0.40억까지 줄였고 아직 0은 아님

제시가 계수는 미검증

0.20 / 0.15 / 0.10은 도메인 판단에 따른 규칙이지 학습된 값이 아님. 실제 협상 결과가 쌓여야 검증 가능



SalaryCast AI

KBO FA 연봉 예측 · 구단별 제시가 추천

데모 `stovelens-ai.streamlit.app`

코드 `github.com/jjssspark/SalaryCast_AI`